

Lab 2_Part 1: Introduction to Logistic regression on Iris Dataset Classification

Overview

The purpose of this lab is to introduce you to the process of building a machine learning models (here, Logistic regression) using the famous Iris dataset. This dataset is a classic in the field of data science and is commonly used for beginners to practice data analysis and classification techniques.

Data Description

The Iris dataset is a multivariate dataset introduced by the British biologist and statistician Ronald Fisher. It consists of 150 samples from three species of Iris flowers:

- **Iris setosa**
- **Iris versicolor**
- **Iris virginica**



figure : three flowers from Iris species

Each sample includes four features:

- **Sepal length** (in cm)
- **Sepal width** (in cm)
- **Petal length** (in cm)
- **Petal width** (in cm)

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Features
(attributes, measurements, dimensions)

Class labels
(targets)

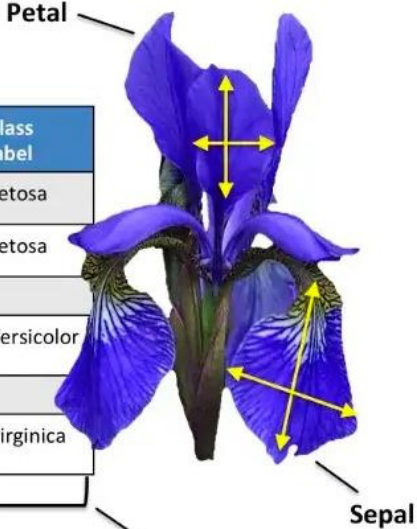


Table 1 : features extraction from data sample

These features represent the physical dimensions of the flowers. The goal is to use these features to classify each sample into one of the three species.

Let's begin ...

Section 1: Data Presentation

Load the Iris Dataset:

1. Open a Jupyter Notebook
2. In a new notebook, load the Iris dataset using the following code:

```
from sklearn.datasets import load_iris  
import pandas as pd
```

```
# Load the Iris dataset  
iris = load_iris()  
# Convert to DataFrame  
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)  
iris_df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)  
  
# Display the first few rows of the dataset  
iris_df.head()
```

Description of the Dataset:

The Iris dataset contains 150 samples from three species of Iris flowers: Iris setosa, Iris versicolor, and Iris virginica.

Each sample has four features: sepal length, sepal width, petal length, and petal width.

The target variable (species) indicates the class of the flower.

Section 2: Data Exploration with ydata_profiling

Generate a Profiling Report:

Use ydata_profiling to generate a detailed report of the dataset:

```
from ydata_profiling import ProfileReport

# Generate the profiling report
profile = ProfileReport(iris_df, title="Iris Dataset Profiling Report")
profile.to_notebook_iframe()
```

Analyze the Report:

Review the profiling report to understand the data better. Pay attention to the distribution of each feature and any potential correlations between features.

Pairplot:

Using [seaborn.pairplot](#) function try to plot the pairplot for the iris dataset.

NB: Make sure you've enabled inline plotting at the start of your notebook using :

```
%matplotlib inline
```

Section 3: Binary Logistic Regression — Virginica vs Not Virginica

Objectives

- Build a binary target: **Virginica (1)** vs **Not Virginica (0)**
- Train, evaluate, and interpret a **binary logistic regression**
- Inspect coefficients, probabilities, and confusion matrix
- Visualize a 2-feature decision boundary

Data loading & target engineering

Build a model that predicts whether a flower is *Iris virginica* or not.

```
# Create a copy and add is_virginica flag
virginica_df = iris_df.copy()
virginica_df["is_virginica"] = (iris.target == 2).astype(int)

# Separate features and label
X = virginica_df[iris.feature_names]
y = virginica_df["is_virginica"]

# Display rows
virginica_df.head(150)
```

Questions

1. How many samples belong to class 1 (Virginica) vs class 0 (Not Virginica)?
2. Why is it useful to convert the target to 0/1 for binary classification?

Train/test split (with stratification)

Split your dataset into **training** and **test** sets while keeping the same proportion of each class ("stratification").

```
# Train_split with stratification
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42,
                                                    stratify=y)
Diln y_train.value_counts(), y_test.value_counts()
```

Questions

3. What does `stratify=y` ensure during the split? Why is it useful to stratify a dataset when splitting it into training and test sets?
4. What are the class counts in train vs test datasets?

Feature scaling

Standardize your training and test data so that every feature is on the same scale.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)
X_train_std = scaler.transform(X_train)
X_test_std = scaler.transform(X_test)
```

Binary logistic regression on standardized features

Fit a binary logistic regression on standardized features

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=500, solver="lbfgs")
logreg.fit(X_train_std, y_train)

print("Coefficients (per feature):", dict(zip(iris.feature_names, logreg.coef_[0])))
print("Intercept:", logreg.intercept_[0])
```

Questions

5. Which features have **positive** coefficients? What does a positive sign mean here?
6. Which single feature appears most predictive of Virginica?

Predict labels and probabilities

Evaluate the performance of a **logistic regression classifier** that predicts whether an iris flower is *Virginica* or not.

```
# Import necessary libraries
import numpy as np
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Use the trained logistic regression model (logreg) to make predictions on the
standardized test set
y_pred = logreg.predict(X_test_std)

# Compute the predicted probabilities for the positive class (Virginica = 1)
# predict_proba returns probabilities for all classes;[:, 1] selects the column for class 1
y_proba = logreg.predict_proba(X_test_std)[:, 1]

# Evaluate model performance

# Print the overall accuracy of the model
print("Accuracy:", accuracy_score(y_test, y_pred))

# Print a detailed classification report (precision, recall, f1-score, support) for each class
print(classification_report(y_test, y_pred, target_names=["Not Virginica", "Virginica"]))

# Print the confusion matrix to see the distribution of correct and incorrect predictions
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
```

Questions

- Interpret the confusion matrix: how many **false negatives** (Virginica predicted as not)?
- When might **recall** for Virginica be more important than precision (give a scenario)?

Threshold tuning

Evaluate how the precision, recall, and F1-score of a classifier vary as you change the decision threshold on the predicted probabilities (`y_proba`).

```
# Import the function to compute precision, recall, F1-score, and support
from sklearn.metrics import precision_recall_fscore_support

# Define a helper function to evaluate performance at a specific probability threshold
def evaluate_at(th):
    # Convert predicted probabilities (y_proba) into binary class predictions
    # Any sample with probability >= threshold is assigned to the positive class (1)
    y_hat = (y_proba >= th).astype(int)

    # Compute precision, recall, F1-score, and support for the predictions
    # 'average="binary"' means we focus on the positive class metrics only
    p, r, f1, _ = precision_recall_fscore_support(y_test, y_hat, average="binary")

    # Return the threshold along with its precision, recall, and F1-score
    return th, p, r, f1

# Evaluate the model at different probability thresholds
for th in [0.3, 0.5, 0.7]:
    # Call the evaluation function for the current threshold
    th, p, r, f1 = evaluate_at(th)

    # Print out the threshold and corresponding metrics, nicely formatted
    print(f"Threshold={th:.2f} Precision={p:.3f} Recall={r:.3f} F1={f1:.3f}")
```

Question

11. Pick a use case where **high recall** for Virginica is critical.
12. Which threshold would you choose and why?

Section 4: Classification Task

In this section, we will use multinomial logistic regression as a classification model to assign new observations to one of the three iris species (classes) : Virginica, Setosa or Versicolor, using all features (Sepal length (in cm), Sepal width (in cm), Petal length (in cm), Petal width (in cm)).

Multinomial logistic regression is the extension of logistic regression to handle more than two classes directly rather than reducing the problem to multiple binary classifications.

Split the Data into Training and Testing Sets:

1. Split the dataset iris to train and test the classification model:

TO DO : preprocess the dataset. Link :

<https://scikit-learn.org/stable/modules/preprocessing.html>

Train a Classification Model:

1. Train a logistic regression classifier

```
model = LogisticRegression(  
    max_iter=200, solver="lbfgs", multi_class="multinomial")
```

```
solver={'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'},  
default='lbfgs'
```

solver is the algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, all solvers except 'liblinear' minimize the full multinomial loss;

2. Evaluate the model using the test data.

```
Do not forget to import classification_report and accuracy_score  
from sklearn.metrics import classification_report, accuracy_score
```

3. Interpret the classification report and accuracy score.

4. Prints your trained logistic regression parameters with readable labels using the following code :

Dilmi & Charrad

```
#Captures the feature (column) names from your X DataFrame as a Python list, to label
coefficients by feature.
feature_names = X.columns.to_list()
#Gets the unique target labels from y and sorts them, producing a stable class order to
#label rows (one row per class).
class_names = model.classes_
# Builds a table of the model's coefficients where:
#rows = classes (using class_names)
#columns = features (using feature_names)
#The values are the weights learned by the model.
#If you trained on standardized features, these are standardized-space coefficients.

coef_df = pd.DataFrame(model.coef_, index=class_names, columns=feature_names)

#Builds a Series of the intercepts (one per class) labeled by class_names.

intercepts = pd.Series(model.intercept_, index=class_names)
print("Standardized-space coefficients (rows=classes, cols=features):")
print(coef_df)
print("Standardized-space intercepts (per class):")
print(intercepts)
```

5. Here's a new observation:

New observation (in original Iris units(cm)):

- sepal length (cm): **6.1**
- sepal width (cm): **2.8**
- petal length (cm): **5.0**
- petal width (cm): **1.9**

👉 Based on your trained model above, **to which class would this be assigned? Why ?**

If your model was trained on **standardized features**, use the same `scaler` to transform before predicting.

Dilmi & Charrad


```
new_obs = pd.DataFrame({
    "sepal length (cm)": 6.1,
    "sepal width (cm)": 2.8,
    "petal length (cm)": 5.0,
    "petal width (cm)": 1.9
})

# If the model was trained on standardized features:
new_obs_std = scaler.transform(new_obs[X.columns])

pred_class = model.predict(new_obs_std)[0]
proba = pd.Series(model.predict_proba(new_obs_std)[0],
index=model.classes_.sort_values(ascending=False))

print("Predicted class:", pred_class)
print("Class probabilities:\n", proba)
```

Section 5: Save the Notebook and Share

1. Save the notebook by selecting File > Save As in the Jupyter Notebook interface.
2. Export the notebook to a PDF (Hint: install Latex to facilitate the conversion).
3. Submit both PDF and notebook.

TO go further :

Now that you're familiar with the Logistic Regression model. The next lab is a more complex problem that deals with both numerical and categorical data types.

Lab 2_Part2: Multi-Class Classification using One-vs-All Logistic Regression and k- Nearest Neighbors on Forest Cover Type Dataset

Objective

In this lab, you will apply both **One-vs-All logistic regression** and **k-Nearest Neighbors (k-NN)** to classify forest cover types using the UCI Forest Cover Type Dataset. You will also analyze the dataset using `ydata_profiling` before preprocessing and evaluate both models to compare their performance.

One-vs-all

One-vs-all (OvA) logistic regression—also called **one-vs-rest (OvR)**—is a strategy to handle multi-class classification using **binary** logistic regression models.

How it works

1. For K classes, train K separate binary models.
 - Model k: “class k” = 1 vs. “all other classes” = 0.
2. At prediction time, compute each model’s score/probability $P(y=k|x)$
3. Pick the class with the highest score/probability.

Dataset Information

- **Name:** Forest Cover Type Dataset
- **Classes:** 7 forest cover types (Spruce/Fir, Lodgepole Pine, etc.)
- **Features:** 54 cartographic attributes (such as elevation, slope, soil type, etc.)
- **Size:** 581,012 rows

Step 1: Setup

1. **Import Libraries:** Ensure that you have the necessary libraries installed and imported:
 - `ydata_profiling` for dataset profiling

- numpy, pandas, matplotlib, seaborn, and sklearn for machine learning and data visualization.
- 2. **Load the Dataset:** Download the dataset from [UCI Machine Learning Repository](#), from Kaggle or load the data directly using pandas.

Step 2: Dataset Profiling using ydata_profiling

1. **Generate a Data Profile Report:** Before performing any preprocessing, create a profile report using ydata_profiling to explore the dataset. This will help identify missing values, outliers, correlations, and distributions.
2. **Analyze the Report:**
 - **Missing Values:** Check if there are any missing values.
 - **Outliers:** Identify potential outliers that might affect the model.
 - **Correlations:** Look for highly correlated features that could be redundant.
 - **Class Balance:** Examine the distribution of the target classes to see if there is class imbalance.

Discussion Points :

- What are the key takeaways from the profile report?
- Are there any features that need to be transformed or removed?
- Is there any noticeable class imbalance in the target variable?

Step 3: Data Preprocessing

1. **Feature Engineering:** Based on the findings from the profile report, proceed with splitting the features (X) and target (y).
2. **Split the Data:** Split the dataset into training and test sets (80/20 split).
3. **Standardize or normalize the Data:** Standardize or transform the features using the right transformation from scikit-learn.preprocessing (eg. StandardScaler) for better convergence of logistic regression. This will also improve the performance of distance-based models like k-NN.

Step 4: Model Training and Evaluation

A. One-vs-All Logistic Regression

1. **Fit Logistic Regression:** Train a logistic regression model using the One-vs-All (OvR) strategy.

2. **Make Predictions:** Use the trained model to predict the forest cover types on the test data.
3. **Evaluate Logistic Regression:** Use the confusion matrix and classification report to evaluate the model's performance.
4. **Visualize the Confusion Matrix:** Create a heatmap of the confusion matrix to visualize how well the logistic regression model performs.

B. k-Nearest Neighbors (k-NN)

1. **Fit k-NN Model:** Train a k-NN model on the training data. Use `n_neighbors=5` as a starting point, but you are encouraged to experiment with different values of `k`.
2. **Make Predictions:** Use the trained k-NN model to predict the forest cover types on the test data.
3. **Evaluate k-NN:** Use the confusion matrix and classification report to evaluate the model's performance.
4. **Visualize the Confusion Matrix:** Create a heatmap of the confusion matrix to visualize how well the k-NN model performs.

Step 5: Analysis and Discussion

1. **Compare the Results:**
 - Discuss the performance of both logistic regression and k-NN using their respective classification reports and confusion matrices.
 - Which model performed better for this dataset, and why?
 - Were there specific classes (forest cover types) that were harder to classify for one model compared to the other?
2. **Hyperparameter Tuning:**
 - For k-NN, you are encouraged to experiment with different values of `k` (e.g., `k=3`, `k=7`, `k=9`) and observe how it impacts the model's accuracy.
 - For logistic regression, you can try adjusting regularization strength (`C`) or using different solvers like `liblinear` or `lbfgs`.
3. **Cross-validation:** Implement cross-validation to evaluate the performance of both models more robustly.

Bonus Challenges

1. **Dimensionality Reduction:** Apply **PCA** or **LDA** to reduce the feature space, then train both models again and see if performance improves.
2. **Feature Selection:** Based on the ydata_profiling report, remove redundant or highly correlated features and re-evaluate both models.
3. **Handling Class Imbalance:** If the dataset shows class imbalance, experiment with class balancing techniques such as **SMOTE** or adjusting class weights.

Deliverables

- Submit your code from a Jupyter notebook as pdf file and notebook.